

AI-Powered Debt Relief & Financial Recovery Platform

Project Implementation Guide

Built with FastAPI, SQLAlchemy, SQLite, React (Vite), and the Google Gemini API

Project Overview

The AI-Powered Debt Relief & Financial Recovery Platform is an intelligent web application developed to simplify and automate debt management and financial recovery for borrowers. The platform enables users to manage loan details, analyze financial health, generate AI-powered negotiation strategies, and receive adaptive settlement recommendations through a secure, modern digital environment.

Built using React.js, FastAPI, Python, SQLite, SQLAlchemy ORM, and the Google Gemini API, the system provides a responsive, scalable, borrower-friendly interface with intelligent financial processing and real-time AI-assisted decision support.

Scenario 1: AI-Powered Settlement Recommendation

A borrower adds loan details including outstanding amount, EMI, overdue duration, and monthly income. The system analyzes the financial profile and generates personalized settlement recommendations along with a debt-stress analysis and financial-health insight.

Scenario 2: Intelligent Negotiation Letter Generation

A borrower selects a loan account and requests a negotiation strategy. The platform uses the Google Gemini API to generate professional, lender-specific negotiation emails and settlement request letters based on the borrower's financial condition and repayment capability.

Scenario 3: Financial Health Tracking & Loan Analysis

A borrower accesses the dashboard to monitor financial health metrics such as monthly surplus, EMI ratio, debt-stress level, and settlement percentages. The platform also maintains AI-generated negotiation history.

Problem Statement

Borrowers who fall behind on loan payments rarely have access to structured guidance. In practice, most people managing overdue debt rely on one of three flawed approaches:

- Doing nothing and letting the account slip further into default, damaging their credit score.
- Contacting the lender directly with an emotional, unstructured message that undersells their situation or overstates what they can actually afford.
- Paying for expensive third-party debt-settlement agencies that charge high fees for advice that could be systematized.

None of these approaches give the borrower an objective, numbers-based view of how serious their situation actually is, or what a fair settlement offer would look like given their specific income and overdue history.

Solution Approach

The platform closes this gap by combining two complementary techniques:

- A deterministic, rule-based financial engine that turns raw loan numbers (outstanding amount, EMI, overdue months, income) into an objective debt-stress score and a defensible settlement percentage — the same calculation every time, with no hallucination risk.
- A generative AI layer (Google Gemini) that takes the output of that engine and turns it into a professional, ready-to-send negotiation letter or email, personalized to the borrower's lender and financial profile.

Keeping these two layers separate is a deliberate design decision: the numbers borrowers rely on for their negotiation strategy come from transparent, auditable logic, while the AI is only ever used for the part it's actually good at — natural language generation — never for deciding the settlement math itself.

Objectives

Centralized Debt Management

Provide a single platform to manage:

- Loan accounts and borrower profiles
- Outstanding balances, EMI, and overdue tracking
- AI-generated settlement recommendations
- Negotiation letter history

Automation of Financial Analysis

Streamline:

- EMI-to-income ratio calculation
- Debt-stress scoring
- Settlement percentage recommendation
- Negotiation letter drafting

AI-Driven Financial Intelligence

Enable:

- Rule-based debt-stress analysis
- Gemini-generated negotiation letters and emails
- Financial health insights in plain English

Security & Reliability

Ensure:

- Environment-variable-based API key management (never hard-coded)
- A working template fallback when no AI key is configured
- Input validation on every API request via Pydantic schemas

System Architecture

The application follows a standard three-tier architecture: a React single-page application (presentation layer), a FastAPI service (application/business logic layer), and a SQLite database (data layer). Each tier is independently deployable — the frontend to Vercel, the backend to Render.

Layer	Technology	Responsibility
Presentation	React 18 + Vite, React Router	Renders the Dashboard, Add Loan, and Loan Detail pages; manages client-side routing and form state
API / Application	FastAPI, Pydantic, Uvicorn	Exposes REST endpoints, validates requests, orchestrates the financial engine and Gemini service
Business Logic	Python (services/)	Rule-based debt-stress scoring, settlement recommendation, Gemini prompt construction and fallback templating
AI Layer	Google Gemini API (gemini-1.5-flash)	Generates natural-language negotiation letters and emails grounded in the computed financial analysis
Data	SQLite + SQLAlchemy ORM	Persists loan accounts and negotiation history; auto-creates schema on first run
Hosting	Render (backend), Vercel (frontend)	Production deployment targets for each independently-deployable tier

Request flow for a typical action, such as generating a negotiation letter, looks like this:

- 1. The React frontend sends a POST request to `/negotiation/generate` with the loan ID, letter type, and tone.
- 2. FastAPI validates the request body against the `NegotiationRequest` Pydantic schema.
- 3. The negotiation router loads the loan from SQLite via SQLAlchemy, then calls the financial-analysis engine to compute the current debt-stress score and settlement recommendation.
- 4. Those results are passed into the Gemini service, which builds a grounded prompt and calls the Gemini API (or falls back to the template generator).
- 5. The generated letter is saved to the `negotiation_history` table and returned to the frontend, which renders it immediately without a page reload.

Phase 1: Requirement Analysis and Planning

Understanding the Business Requirement

Borrowers struggling with debt typically rely on scattered spreadsheets, phone calls, and generic advice, which leads to:

- No clear picture of debt-stress level
- Inconsistent or emotionally-written negotiation letters
- No structured settlement strategy
- Poor tracking of EMI burden versus income

The project requires a centralized system that can:

- Track all loan accounts in one place
- Standardize financial-health scoring
- Automate settlement recommendations
- Use AI to draft professional negotiation letters

Why This Stack

FastAPI, SQLite, and SQLAlchemy were selected because they offer:

- A lightweight, file-based database with zero server setup
- Automatic request validation and interactive API docs (Swagger UI)
- A clean ORM layer that maps directly to Python classes
- Fast local development with instant reload

React with Vite was selected for the frontend because it offers a fast dev server, component reuse across pages, and a simple build step for deployment to Vercel.

Project Scope

The project includes building a full-stack debt-recovery platform without any manual, low-code, or drag-and-drop tooling — all business logic is implemented in Python and React.

The application supports the full recovery lifecycle:

Loan Intake → Financial Analysis → Settlement Recommendation → Negotiation Letter → Dashboard Tracking

Phase 2: Backend Development — Configuration & Data Layer

Milestone 1: Environment Setup

Activity 1: Install Prerequisites

1. Install Python 3.10 or above from python.org (Mac users can also use: brew install python).
2. Install Node.js 18 or above from nodejs.org (needed for the React frontend).
3. Install VS Code from code.visualstudio.com.
4. Install the Python and ES7+ React/Redux snippets extensions inside VS Code.

Activity 2: Create a Gemini API Key

5. Go to <https://aistudio.google.com/app/apikey>.
6. Sign in with a Google account and click Create API Key.
7. Copy the key — this will be pasted into the backend .env file.

Note: The application works even without this key. If no key is configured, the negotiation-letter feature automatically falls back to a template-based generator, so the app is always fully functional.

Milestone 2: Project Structure & Database Connection

Activity 1: Create the Backend Folder Structure

Inside the project root, the backend is organized as follows:

```
backend/  
  main.py  
  database.py  
  models.py  
  schemas.py  
  routers/  
    loans.py  
    negotiation.py  
    dashboard.py  
  services/  
    financial_analysis.py  
    gemini_service.py
```

Activity 2: Configure the SQLite Database Connection

8. Create database.py and set up the SQLAlchemy engine, session, and declarative Base.
9. Point DATABASE_URL at a local SQLite file: `sqlite:///./debt_relief.db`
10. Pass `connect_args={"check_same_thread": False}` so SQLite works correctly with FastAPI's threaded request handling.
11. Define a `get_db()` dependency that yields a session and closes it after each request.

```
engine = create_engine(  
    DATABASE_URL, connect_args={"check_same_thread": False}  
)  
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)  
Base = declarative_base()
```

Milestone 3: Data Model Creation

Activity 1: Creating the Loan Model

Using SQLAlchemy's declarative ORM, define a Loan class mapped to the loans table with the fields listed in the data dictionary below.

```
class Loan(Base):
    __tablename__ = "loans"
    id = Column(Integer, primary_key=True, index=True)
    borrower_name = Column(String, nullable=False)
    lender_name = Column(String, nullable=False)
    loan_type = Column(String, default="Personal Loan")
    outstanding_amount = Column(Float, nullable=False)
    emi = Column(Float, nullable=False)
    overdue_months = Column(Integer, default=0)
    monthly_income = Column(Float, nullable=False)
    interest_rate = Column(Float, default=0.0)
    created_at = Column(DateTime, default=datetime.utcnow)
    negotiations = relationship("NegotiationHistory", back_populates="loan")
```

Activity 2: Creating the NegotiationHistory Model

Define a NegotiationHistory class mapped to the negotiation_history table, with a foreign key back to loans.id and a relationship() so each loan can list its past negotiation letters.

```
class NegotiationHistory(Base):
    __tablename__ = "negotiation_history"
    id = Column(Integer, primary_key=True, index=True)
    loan_id = Column(Integer, ForeignKey("loans.id"), nullable=False)
    letter_type = Column(String, default="settlement_request")
    generated_text = Column(Text, nullable=False)
    debt_stress_level = Column(String, nullable=True)
    settlement_percentage = Column(Float, nullable=True)
    created_at = Column(DateTime, default=datetime.utcnow)
    loan = relationship("Loan", back_populates="negotiations")
```

Data Dictionary

Object	Field	Data Type	Notes
Loan	id	Integer (PK)	Auto-generated primary key
Loan	borrower_name	String	Required
Loan	lender_name	String	Required
Loan	loan_type	String	Personal, Credit Card, Auto, Education, Home, Business
Loan	outstanding_amount	Float	Must be greater than 0
Loan	emi	Float	Monthly EMI, must be greater than 0
Loan	overdue_months	Integer	Defaults to 0
Loan	monthly_income	Float	Required, must be greater than 0
Loan	interest_rate	Float	Optional, defaults to 0
NegotiationHistory	id	Integer (PK)	Auto-generated primary key
NegotiationHistory	loan_id	Integer (FK)	References Loan.id
NegotiationHistory	letter_type	String	settlement_request or negotiation_email
NegotiationHistory	generated_text	Text	AI or template-generated letter body
NegotiationHistory	debt_stress_level	String	Low, Moderate, High, Critical

NegotiationHistory	settlement_percentage	Float	Recommended settlement % at time of generation
--------------------	-----------------------	-------	--

Milestone 4: Request & Response Schemas

Activity 1: Define Pydantic Schemas

12. Create schemas.py.
13. Define LoanCreate for incoming POST /loans requests, with field-level validation (e.g. Field(gt=0) on money fields).
14. Define LoanOut for API responses, using ConfigDict(from_attributes=True) so it maps directly from the ORM object.
15. Define FinancialAnalysis, NegotiationRequest, NegotiationOut, and DashboardSummary for the remaining endpoints.

Note: Pydantic validation runs automatically on every request — if a borrower submits a negative EMI or a missing field, FastAPI returns a 422 error before the request ever reaches business logic.

Milestone 5: Financial Analysis Engine (Business Logic)

This is the rule-based core that powers Scenario 1. It has no external dependencies, so it runs instantly and deterministically.

Activity 1: EMI Ratio & Monthly Surplus

16. Create `services/financial_analysis.py`.
17. Calculate $\text{emi_ratio} = (\text{EMI} \div \text{monthly income}) \times 100$.
18. Calculate $\text{monthly_surplus} = \text{monthly income} - \text{EMI}$.

Activity 2: Debt-Stress Score

19. Blend three weighted signals into a 0–100 score: EMI ratio (60% weight), overdue duration (up to 40 points), and a flat penalty if monthly surplus is negative.
20. Map the score to a stress level: Low (< 30), Moderate (30–54), High (55–79), Critical (80+).

Activity 3: Settlement Percentage Recommendation

21. Start from a 75% base settlement offer.
22. Reduce the percentage as overdue months increase (up to –30 points).
23. Reduce further based on stress level: Moderate –5, High –12, Critical –20.
24. Clamp the final result between 35% and 75% of the outstanding amount.

Activity 4: Plain-English Insight

Generate a short, borrower-facing insight paragraph tailored to the stress level — e.g. recommending immediate lender contact for Critical accounts, or highlighting a healthy surplus for Low-stress accounts.

Milestone 6: Gemini AI Integration

Activity 1: Configure the Gemini Client

25. Create `services/gemini_service.py`.
26. Load `GEMINI_API_KEY` from the `.env` file using `python-dotenv`.
27. If a key is present, initialize `genai.GenerativeModel("gemini-1.5-flash")`; otherwise leave the model unset.

Activity 2: Build the Negotiation Letter Prompt

28. Construct a prompt that includes the borrower's name, lender, loan type, outstanding amount, EMI, overdue duration, income, EMI ratio, debt-stress level, and the recommended settlement amount.
29. Instruct the model to keep the letter under 300 words, respectful in tone, and to close with a clear call to action.

Activity 3: Template Fallback

30. If no API key is configured, or the Gemini call fails for any reason, generate the same letter from a deterministic Python string template using the same financial inputs.

Note: This fallback guarantees Scenario 2 always produces a usable letter, even completely offline.

Milestone 7: API Endpoints

Activity 1: Loan Endpoints (*routers/loans.py*)

- POST /loans — create a loan account
- GET /loans — list all loan accounts
- GET /loans/{id} — retrieve a single loan
- GET /loans/{id}/analysis — loan plus computed financial-health analysis
- DELETE /loans/{id} — remove a loan account

Activity 2: Negotiation Endpoints (*routers/negotiation.py*)

- POST /negotiation/generate — run the financial-analysis engine, call the Gemini service, and store the result in negotiation_history
- GET /negotiation/history/{loan_id} — return all past letters for a loan, most recent first

Activity 3: Dashboard Endpoint (*routers/dashboard.py*)

- GET /dashboard/summary — aggregate total loans, total outstanding, total monthly EMI, average EMI ratio, and a count of Critical-stress accounts, alongside every loan's individual analysis

Milestone 8: Assembling the FastAPI Application

31. In main.py, call Base.metadata.create_all(bind=engine) on startup so the SQLite tables are created automatically the first time the app runs.
32. Add CORSMiddleware so the React dev server (localhost:5173) can call the API on localhost:8000.
33. Include all three routers with app.include_router(...).
34. Run the server: uvicorn main:app --reload --port 8000
35. Verify the API by opening the interactive Swagger docs at <http://localhost:8000/docs>.

API Reference

The complete set of endpoints exposed by the backend, all documented automatically by FastAPI's built-in OpenAPI/Swagger UI:

Method	Endpoint	Description
POST	/loans	Create a new loan account from borrower-supplied details
GET	/loans	List every loan account, most recently created first
GET	/loans/{id}	Retrieve a single loan's raw details
GET	/loans/{id}/analysis	Loan details plus the computed financial-health analysis (EMI ratio, surplus, stress score/level, settlement recommendation, insight)
DELETE	/loans/{id}	Delete a loan account and its associated negotiation history
POST	/negotiation/generate	Run the financial engine, call Gemini (or the fallback template), persist the result, and return the generated letter
GET	/negotiation/history/{loan_id}	List every previously generated letter for a loan, most recent first
GET	/dashboard/summary	Aggregate statistics across all loans plus each loan's individual analysis, used to populate the Dashboard page

GET	/health	Lightweight health-check endpoint used by the hosting platform
-----	---------	--

Phase 3: Frontend Development (React + Vite)

Milestone 9: Frontend Project Setup

36. Create the frontend/ folder with package.json, vite.config.js, and index.html.
37. Add react-router-dom for client-side routing between the Dashboard, Add Loan, and Loan Detail pages.
38. Run npm install, then npm run dev to start the Vite dev server on http://localhost:5173.

Milestone 10: Design System

A custom "ink-ledger" visual identity was built rather than a generic template — a deep ink-teal background with gold accents, a serif display face for headings, and a monospace face for all financial figures, evoking a passbook / financial ledger.

- Fonts: Fraunces (display), Inter (body), IBM Plex Mono (figures)
- Palette: ink background, gold accent, sage/amber/rust status colors for Low/Moderate/High/Critical
- Signature element: a circular arc gauge that visually reads the debt-stress score, used on both the loan-detail page

Milestone 11: Reusable Components

Activity 1: Navbar

A persistent top navigation bar linking to the Dashboard and Add Loan pages, with the active route highlighted.

Activity 2: LoanCard

Displays a lender name, loan type, outstanding amount, EMI ratio, overdue months, and a color-coded stress badge. Wrapped in a Link that opens the loan's detail page.

Activity 3: StatCard

A single labeled metric used in the dashboard's top summary strip (total loans, total outstanding, monthly EMI load, critical accounts).

Activity 4: StressGauge

An SVG arc gauge (270°) that animates to the loan's 0–100 debt-stress score and colors itself according to stress level.

Milestone 12: Pages

Activity 1: Dashboard Page (Scenario 3)

39. On mount, call GET /dashboard/summary.
40. Render the stat strip, followed by a grid of LoanCard components.
41. Show an empty state with a call-to-action link to Add Loan when no loans exist yet.

Activity 2: Add Loan Page (Scenario 1 input)

42. Build a controlled form covering borrower name, lender name, loan type, interest rate, outstanding amount, EMI, overdue months, and monthly income.
43. On submit, call POST /loans, then redirect to the new loan's detail page.

Activity 3: Loan Detail Page (Scenarios 1 & 2)

44. On mount, call GET /loans/{id}/analysis and GET /negotiation/history/{id} in parallel.
45. Render the StressGauge, a key-value breakdown of the financial profile, and the plain-English insight box.
46. Provide letter-type and tone selectors, and a Generate Letter button that calls POST /negotiation/generate.
47. Display the generated letter and prepend it to the negotiation-history list without a full page reload.

Milestone 13: Connecting Frontend to Backend

48. Create src/api.js as a thin fetch wrapper pointed at http://localhost:8000, with one function per backend endpoint.
49. Centralize error handling so a non-2xx response throws with the backend's detail message, which pages display in an error banner.

```
export const api = {
  getDashboardSummary: () =>
    fetch(`${BASE_URL}/dashboard/summary`).then(handleResponse),
  generateNegotiationLetter: (data) =>
    fetch(`${BASE_URL}/negotiation/generate`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(data),
    }).then(handleResponse),
};
```

Phase 4: Testing, Version Control & Deployment

Milestone 14: End-to-End Testing

Activity 1: Scenario 1 — Settlement Recommendation

- 50. Add a loan with a high EMI-to-income ratio and a few overdue months.
- 51. Confirm the debt-stress level, score, and settlement recommendation appear correctly on the detail page.

Activity 2: Scenario 2 — Negotiation Letter

- 52. From the loan detail page, choose a letter type and tone, then click Generate Letter.
- 53. Confirm the letter references the correct lender, amounts, and settlement percentage.
- 54. Confirm the letter is saved and appears in the negotiation history list.

Activity 3: Scenario 3 — Dashboard Tracking

- 55. Return to the dashboard and confirm the summary statistics update to reflect all loans added so far.
- 56. Add a second loan with a critical stress level and confirm the Critical Accounts counter increments.

Test Case Matrix

A representative set of test cases used to validate the financial-analysis engine and API layer end-to-end:

Input	Expected Behavior	Validates
EMI = 60% of income, 0 months overdue	Moderate-to-High stress level, positive but thin monthly surplus	EMI-ratio weighting in the stress score formula
EMI exceeds monthly income, 6 months overdue	Critical stress level, negative monthly surplus, settlement % clamped near the 35% floor	Overdue-duration penalty and negative-surplus penalty
Low EMI relative to income, 0 months overdue	Low stress level, settlement % near the 75% ceiling	Healthy-profile path through the scoring logic
outstanding_amount = 0 or negative	422 Unprocessable Entity from FastAPI before reaching business logic	Pydantic Field(gt=0) validation on LoanCreate
POST /negotiation/generate with an invalid loan_id	404 Not Found with a clear error message	Router-level existence checks before calling Gemini
Negotiation letter requested with no GEMINI_API_KEY set	A complete, well-formatted letter is still returned via the fallback template	Graceful degradation of the AI layer

The backend was also verified directly against FastAPI's TestClient during development — creating a loan, requesting its analysis, generating a negotiation letter, and pulling the dashboard summary — confirming all three scenarios work correctly end-to-end before the frontend was connected.

Milestone 15: Version Control (Git & GitHub)

57. Add a .gitignore excluding venv/, node_modules/, .env, and any *.db file — these should never be committed.
58. Run git init, git add ., git commit -m "Initial commit", and git branch -M main.
59. Create an empty repository on GitHub, then push with git remote add origin <repo-url> and git push -u origin main.

Note: Never commit the .env file — it holds the Gemini API key. Each teammate should copy .env.example to their own .env locally.

Milestone 16: Deployment

Activity 1: Deploy the Backend to Render

60. Create a new Web Service on Render, pointing at the backend/ folder of the GitHub repository.
61. Set the build command to pip install -r requirements.txt and the start command to uvicorn main:app --host 0.0.0.0 --port \$PORT.
62. Add GEMINI_API_KEY as an environment variable in Render's dashboard (never in code).

Activity 2: Deploy the Frontend to Vercel

63. Import the repository into Vercel and set the frontend/ folder as the project root.
64. Update src/api.js's BASE_URL to point at the deployed Render backend URL.
65. Deploy — Vercel automatically rebuilds on every push to main.

Project Structure Reference

```
debt-relief-platform/
├── backend/
│   ├── main.py
│   ├── database.py
│   ├── models.py
│   ├── schemas.py
│   ├── routers/ (loans, negotiation, dashboard)
│   └── services/ (financial_analysis, gemini_service)
├── frontend/
│   └── src/
│       ├── App.jsx, api.js, index.css
│       ├── components/ (Navbar, LoanCard, StatCard, StressGauge)
│       └── pages/ (Dashboard, AddLoan, LoanDetail)
```

Challenges Faced & Solutions

Challenge: Keeping AI output grounded in real numbers

A generic call to an AI model can produce a plausible-sounding but factually wrong negotiation letter — the wrong settlement amount, an invented interest rate, or a tone that doesn't match the borrower's actual hardship level.

Solution: The financial-analysis engine always runs first and its exact output (EMI ratio, stress level, settlement amount) is injected directly into the Gemini prompt as structured facts. The model is instructed to work only from those numbers, which keeps the generated letter consistent with what the borrower sees on the analysis panel.

Challenge: The feature must work even without an API key

Requiring every developer or evaluator to obtain a Gemini API key before the app runs at all would be a poor experience, especially during a hackathon demo.

Solution: `gemini_service.py` checks for a configured key at import time. If it's missing, or the Gemini call throws an exception for any reason (rate limit, network issue), the service transparently falls back to a deterministic Python string template that uses the exact same financial inputs — so Scenario 2 always produces a complete, usable letter.

Challenge: Designing a settlement formula that is defensible, not arbitrary

A single percentage that ignores overdue duration or debt-stress level would not reflect real negotiation dynamics, where longer-overdue and higher-stress accounts are typically settled for less.

Solution: The settlement percentage starts from a 75% base and is reduced by two independent factors — overdue months (up to -30 points) and stress level (-5 to -20 points) — then clamped between 35% and 75%. Each input's effect on the final number is traceable, which matters when a borrower asks why a particular percentage was recommended.

Challenge: SQLite and FastAPI's threaded request handling

SQLite connections are not thread-safe by default, which can cause errors under FastAPI's default threaded execution model.

Solution: The engine is created with `connect_args={"check_same_thread": False}`, and every request gets its own session via the `get_db()` dependency, which is opened and closed per-request rather than shared globally.

Future Enhancements

- User authentication (JWT) so each borrower only sees their own loan accounts.
- PDF export of the generated negotiation letter for direct download or email attachment.
- EMI-ratio and debt-stress trend charts over time, once loans accumulate a payment history.
- A lender directory with saved contact details, so negotiation emails can be sent directly from the platform.
- Multi-currency support for borrowers outside India.

- A recurring reminder system that nudges borrowers to follow up if a lender hasn't responded within the requested 7–10 business days.

Conclusion

The AI-Powered Debt Relief & Financial Recovery Platform demonstrates how a deterministic, rule-based financial engine and a generative AI layer can be combined responsibly — using the AI strictly for language generation while keeping every financial number transparent and auditable. The result is a full-stack application that takes a borrower from raw loan numbers to a concrete, ready-to-send negotiation strategy in under a minute, built entirely on a free-tier-friendly stack (FastAPI, SQLite, React, and the Gemini API) that is straightforward to run locally and deploy to production.